

Session 11: Regular Expressions, Scripting, and Functional Programming in Python

Detailed Theory

1. Regular Expressions Using Python

- Regular expressions (regex) are patterns used to match character combinations in strings.
- Python provides the `re` module to work with regex.
- Common functions in `re` module:
 - `re.match()` – checks for a match only at the beginning of the string.

- re.search() — searches for the pattern anywhere in the string.
- re.findall() — returns a list of all matches.
- re.sub() — replaces matched parts with a new string.

- **Basic regex syntax:**

- . — any character except newline
- ^ — start of string
- \$ — end of string
- * — 0 or more repetitions

- + – 1 or more repetitions
- ? – 0 or 1 repetition
- {n,m} – between n and m repetitions
- [] – character class, e.g., [a-z]
- \d – digit [0-9]
- \w – word character [a-zA-Z0-9_]
- \s – whitespace

- Example:

python

CopyEdit

```
import re pattern = r"\d{3}-\d{2}-\d{4}" text = "My SSN is 123-45-6789." match = re.search(pattern, text) if match: print("Found:", match.group())
```

2. Scripting in Python

- Python scripts are files containing sequences of Python commands, executed top-to-bottom.
- Scripts can automate tasks like file processing, system operations, web scraping, etc.
- To run a Python script:

bash

CopyEdit

python script.py

- Common scripting practices:
 - Use if `__name__ == "__main__"`: to run code only when script executed directly.
 - Use command-line arguments via `sys.argv` or `argparse` for flexible inputs.
 - Modularize code with functions for reusability.
 - Handle errors using try-except blocks.

-

Example simple script to count lines in a file:

python

CopyEdit

```
import sys
def count_lines(filename):
    with open(filename, 'r') as f:
        return len(f.readlines())
if __name__ == "__main__":
    filename = sys.argv[1]
    print(f"Number of lines: {count_lines(filename)}")
```

3. Functions and Functional Programming

-

Python treats functions as **first-class objects** — they can be assigned, passed, and returned like variables.

-

Supports **functional programming** concepts such as:

-

Higher-order functions: Functions that take other functions as arguments or return them.

-

Lambda functions: Anonymous, inline functions defined with lambda.

-

Map, filter, reduce: Functional tools to process iterables.

-

Examples:

python

CopyEdit

```
# Lambda function square = lambda x: x**2 # map() applies function to all elements nums = [1, 2, 3, 4] squares = list(map(square, nums)) #
filter() selects elements based on condition evens = list(filter(lambda x: x % 2 == 0, nums)) # reduce() accumulates values (needs
functools) from functools import reduce product = reduce(lambda x, y: x * y, nums)
```

-

Benefits: concise code, easier to pass behavior as arguments, better for parallel processing.
